

First-Year Progression Review

Topological Feature Learning for Parameter Estimation in Neyman-Scott Processes; Hessian Spectra During Grokking

Flavio Gualtieri

August 24, 2026

This review reports two disjoint projects from my first year. The first – largely exploratory – concerns the spectrum of the Hessian eigenvalue spectrum, specifically how it relates to the phenomenon of Grokking. The second projects – considerably more substantial – is a study of persistence-based feature learning for parameter estimation in Neyman-Scott point processes. This consists of a synthetic point process simulator, the ParamNet estimator and its architectural ablations, and a comparison of topological vectorizations.

Part I

Topological Feature Learning for Parameter Estimation in Neyman-Scott Processes

1 Introduction

The study of spatial point processes finds applications in astronomy, cell biology, epidemiology, etc (Møller and Waagepetersen, 2004). Many natural phenomena – such as galaxy clustering – are modelled using SPPs, so one can hope to leverage an understanding of these processes to obtain novel insights. The parameters of spatial point processes are of particular interest because they are interpretable scientific quantities. For example, when Neyman-Scott processes are used to model galaxy clustering the parent intensity κ corresponds to the number density of haloes hosting galaxies, the offspring intensity μ is the mean occupancy of a halo, and the cluster scale σ is the spatial extent of the galaxy distribution within a halo REF.

Existing estimation methods typically rely on closed-form expressions for summary statistics such as the K and L functions. Minimum contrast estimation fits parameters by numerically minimizing a discrepancy between the empirical and theoretical curve, most commonly for K or g (Diggle, 2003). Palm likelihood instead builds a composite likelihood from the process’s Palm intensity, developed specifically for Neyman-Scott processes (Tanaka et al., 2008).

These methods share three limitations. First, they rely on closed-form summary statistics, which may not exist (the LGCP-Strauss process of Vihrs (2022) has no tractable density) or may exist but fail the regularity conditions the estimator’s theory needs (the two-step estimator of Waagepetersen and Guan (2009) requires K_ψ to be twice continuously differentiable in ψ , which excludes the Matérn cluster process). Second, they are sensitive to hyperparameters chosen per dataset by rule of thumb, with no held-out quantity indicating whether the choice was good: minimum contrast depends on (r_{max}, q, p) , and Waagepetersen and Guan (2009) report that Thomas estimates of κ and ω are “quite sensitive” to r_{max} and q , being strongly negatively correlated so that the product $\hat{\kappa}\hat{\omega}$ is stable while the individual estimates are not. Finally, they destabilise as the process approaches complete spatial randomness: writing $\lambda = \kappa\mu$ for the expected number of points per unit area and $\nu = \sigma\sqrt{\kappa}$ for the ratio of cluster radius to mean parent spacing, once $\nu \gtrsim 1$ neighbouring clusters overlap and many (κ, μ) pairs with the same product produce nearly the same observable pattern. The theoretical K -function makes this explicit – for a Thomas process $K(t) = \pi t^2 + (1 - e^{-t^2/4\sigma^2})/\kappa$, so as σ grows at fixed t the

second term vanishes and K tends to the Poisson K regardless of κ – and the contrast surface is close to flat along this trade-off, so estimates become sensitive to the starting point.

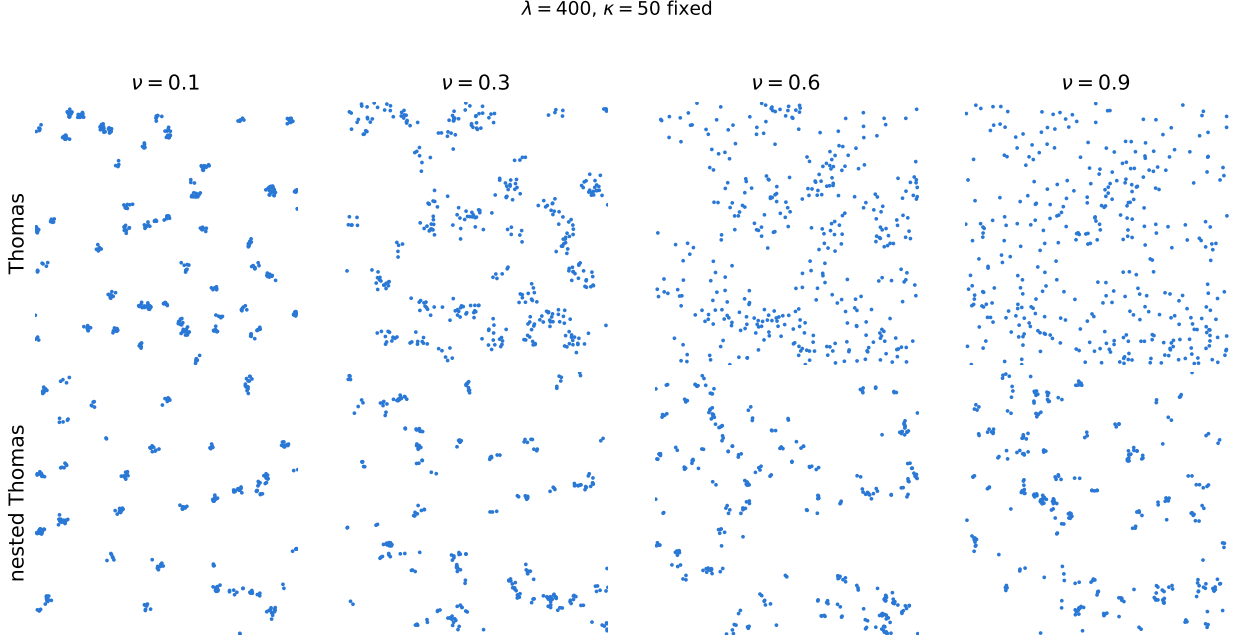


Figure 1: Simulated realisations across the overlap index $\nu \in \{0.1, 0.3, 0.6, 0.9\}$ at fixed λ, κ : Thomas (top) vs. nested Thomas (bottom). One seed per panel, generated with the simulator of Section 3.1.

This prompted me to consider whether persistence-based features could be used in this context, and whether they have advantages over these classical methods: they provide a multi-scale, process-agnostic descriptor of a point cloud that requires neither a closed form nor a choice of summary statistic.

1.1 Problem setup

Let X be a Neyman-Scott process on $\mathbb{R}^d$ with unknown parameter vector $\theta \in \mathbb{R}^{n_{\text{params}}}$. For example, the homogeneous Thomas process has $\theta = (\kappa, \mu, \sigma)$ corresponding to parent intensity, offspring intensity and cluster scale, respectively. We observe a single realization of X on a bounded window $W \subset \mathbb{R}^d$, giving a point cloud $\mathbf{x} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ with $n = |X \cap W|$. The objective is to produce an estimate $\hat{\theta} = \hat{\theta}(\mathbf{x})$ of θ from $\mathbf{x}$ alone. We stress *single realization*: θ is not recovered from repeated draws of X . This is the setting a practitioner faces: one observed pattern and one chance to estimate the generating parameters.

The model is trained on a set of synthetic clouds $\{\mathbf{x}_i, \theta_i\}_{i=1}^{N_{\text{train}}}$ by backpropagating the RMS loss

$$\mathcal{L}(\hat{\theta}_i, \theta_i) = \sqrt{\sum_{j=1}^{n_{\text{params}}} (\hat{\theta}_i^j - \theta_i^j)^2}.$$

The scale of each parameter varies; therefore – if not normalized – the large-scale parameters will dominate this loss. To address this, we train in log-normalized parameter space, mapping $\theta_i \mapsto (\log(\theta_i) - \mu_{\log \theta}) / \sigma_{\log \theta}$, where – abusing notation – $\mu_{\log \theta}, \sigma_{\log \theta}$ denote the mean and standard deviation of the logged parameters over the training set. All losses reported below are in this space, so that $\mathcal{L} \approx 1$ corresponds to chance level.

2 Background

Vihrs (2022) first proposed the use of deep learning for parameter estimation. Yip et al. (2025) employ a similar architecture with persistence images as the input feature, mapping H_0 – H_2 images of an α DTM ℓ filtration of a halo catalogue to (Ω_m, σ_8) ; their targets are cosmological parameters rather than those of the underlying point process, and they do not systematically compare topological summaries. Which summary to use is an important and ongoing question in this investigation, since persistence diagrams

do not live in a Hilbert space and cannot be fed directly to a machine learning model: Adams et al. (2017) propose persistence images, Umeda (2017) Betti curves, Bubenik (2015) persistence landscapes and Chazal et al. (2014) silhouettes, while Reininghaus et al. (2015) avoid vectorization altogether with a kernel on diagrams.

2.1 Neyman-Scott processes

A spatial point process on $W \subseteq \mathbb{R}^d$ is characterised by its intensity function $\rho(u)$, which satisfies $\mathbb{E}[N(W)] = \int_W \rho(u) du$ for $N(W) = |W \cap X|$; when ρ is constant the process is homogeneous, the canonical example being the Poisson process. A Neyman-Scott process is a clustered process built on top of a Poisson; the processes used throughout this work follow the standard construction: expand the window W by an edge buffer to give W_{exp} , so that clusters with parents outside W are not truncated; sample a homogeneous Poisson *parent* process of intensity κ on W_{exp} ; for each parent p draw an offspring count $n_p \sim \text{Poisson}(\mu)$; place each offspring at $p + \epsilon$ with displacement ϵ drawn from a kernel k ; finally discard the parents and retain the offspring falling in W .

Every Neyman-Scott process therefore has the parent and offspring intensities (κ, μ) as parameters, and the choice of kernel supplies the rest: for the *Thomas* process $k = \mathcal{N}(0, \sigma^2 I_d)$, giving $\theta = (\kappa, \mu, \sigma)$; for the Matérn cluster process k is uniform on a ball of radius R ; and for the *nested Thomas* process the parent process at the second step is itself a Thomas process rather than a Poisson one, producing clusters of clusters and a five-parameter target.

2.2 Persistent homology

Persistent homology (PH) tracks a growing family of simplicial complexes built on a point cloud. A *filtration function* f assigns to each subset of points a scale at which it appears in the resulting filtration $\mathcal{F}_r$, subject to a monotonicity condition, and the sublevel sets $\mathcal{F}_r = \{\sigma : f(\sigma) \leq r\}$ form a nested family of complexes indexed by r . The canonical choice is Vietoris-Rips, where a simplex enters at its diameter; the DTM_k filtration (Anai et al., 2020) instead uses each point’s root-mean-square distance to its k nearest neighbours, so a simplex enters according to how densely its neighbourhood is occupied rather than how close its points happen to be. This preserves more information about the local density of the point cloud, which is particularly useful in this context, as the generating parameters of a point process directly affect this density.

As r grows, homological features – connected components (H_0) and loops (H_1) – are *born* and later *die* by merging into older features. Each is recorded as a pair (b, d) of birth and death scales, and the multiset of all such pairs in dimension h is the *persistence diagram* D_h ; the lifetime $d - b$ is often taken as a measure of a feature’s robustness, so points near the diagonal are usually read as noise. Applied to a point pattern, H_0 records the order and scales at which clusters merge and H_1 the gaps and voids the clustering leaves behind, neither requiring a closed form or an assumption about which family generated the pattern. As mentioned in Section 2, persistence diagrams are multisets rather than vectors, so they must be *vectorized* before being used for machine learning; the persistence image (Adams et al., 2017) maps to birth-persistence coordinates $(b, d - b)$, places a weighted Gaussian bump at each point, and integrates the resulting surface over a 64×64 pixel grid.

The principal difference between features based on persistence summaries and those based on classical ... Radially averaged statistics such as g and K collapse a point cloud to a real-valued curve indexed by scale r , and are turned into features by evaluating over $[r_{min}, r_{max}]$ and subsampling to obtain a vector in $\mathbb{R}^n$; this discards multi-scale connectivity and void structure, i.e. *how* the density is distributed across space.

3 ParamNet

The work completed so far consists of the simulator and its validation, the ParamNet model and its architectural ablations, comparison with alternative vectorizations, and benchmarking against existing baselines trained on the same data. TODO: don’t like phrasing.

3.1 Synthetic cloud generation

Since the estimator is trained rather than derived, the simulator plays the role that a closed-form summary statistic plays in a classical method: it defines what the model is asked to learn. Training data are pairs $(\mathbf{x}_i, \theta_i)$ where θ_i is drawn from a *design distribution* over parameter space and $\mathbf{x}_i$ is a

Same cloud, same intermediate value $r = 1.3$

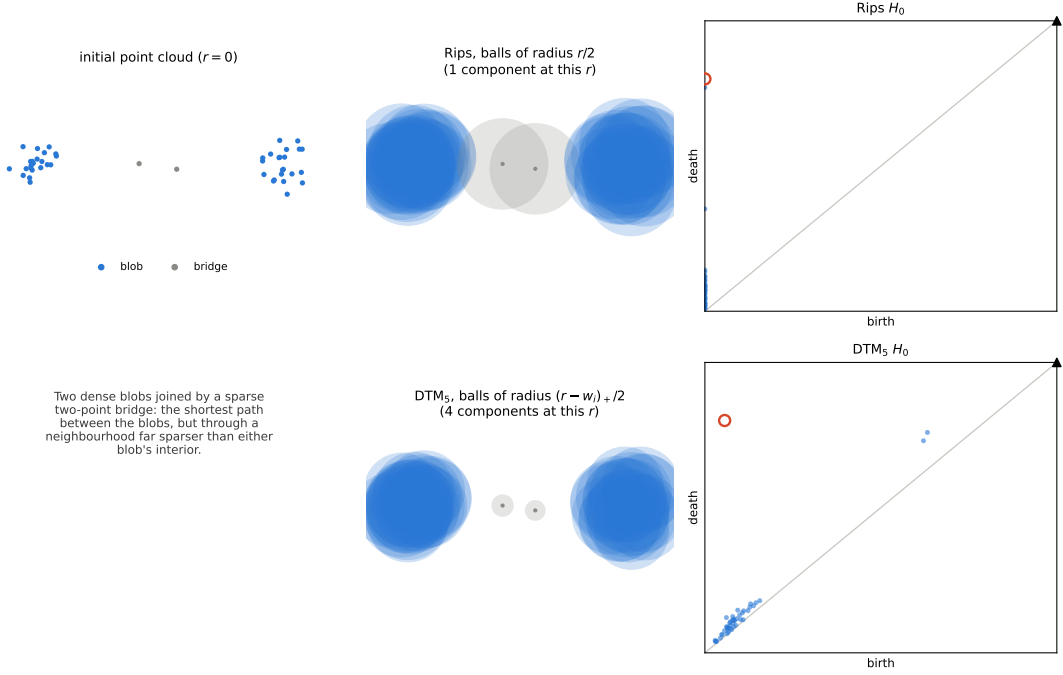


Figure 2: Rips vs. DTM_5 on one clustered cloud (two blobs joined by a sparse two-point bridge), same intermediate filtration value r : Rips already merges the blobs through the bridge (left, top), while DTM_5 delays exactly that merge because the bridge sits in a sparse neighbourhood (left, bottom) – shown in the H_0 diagrams (right) by the highlighted merge event sitting much further from the diagonal’s origin under DTM .

single realization of the corresponding process, generated by the construction of Section 2.1. The design distribution therefore acts as a prior, and the qualitative regimes it covers determine where the trained model is usable.

After adopting this naive approach at first, I found that sampling each parameter uniformly and independently from some interval is inadequate, because the regimes of interest occupy curved subsets of the parameter grid and most of the mass landed on configurations that were near-Poisson or degenerate.

To address this, I moved to sampling in reparameterized coordinates that allow for more direct control of these regimes: the expected intensity $\lambda = \kappa\mu$, the parent intensity κ , and the dimensionless overlap index $\nu = r\sqrt{\kappa}$, where r is the cluster scale of the family in question ($r = \sigma$ for Thomas, $r = \sqrt{\sigma_1^2 + \sigma_2^2}$ for nested Thomas, with the extra coordinate $\rho = \sigma_2/\sigma_1$). Since ν measures cluster radius against mean inter-parent spacing, $\nu \ll 1$ gives well-separated clusters and $\nu \gtrsim 1$ clusters that merge. Draws are log-uniform, since the parameters span orders of magnitude, and are constrained so that $\lambda \in [150, 800]$ (persistence computation scales badly in the number of points), $\kappa \in [15, 120]$ (enough clusters per window to be informative, few enough to remain resolvable), and $\nu \in [0.10, 0.90]$. The upper limit on ν deliberately retains the near-Poisson regime in which classical estimators are least stable, rather than restricting to well-separated clusters. Rejection acts on the design and not on realizations, so it does not distort the conditional law of $\mathbf{x}$ given θ .

3.2 Model: ParamNet

Let $\mathbf{x}$ denote a point cloud, v its vectorized persistence summary, and $e \in \mathbb{R}^{E+1}$ the resulting feature vector, where E denotes the encoder’s embedding dimension and the $+1$ is the appended $\log N(\mathbf{x})$. ParamNet follows the pipeline

$$\mathbf{x} \xrightarrow{\text{Vectorization}} v \xrightarrow{\text{Encoder}} e \xrightarrow{\text{FCNN}} \hat{\theta}. \quad (1)$$

The pipeline shape follows the neural-estimator patterns used in Vihrs (2022) and Yip et al. (2025); what I have added is the application to a more challenging process (nested Thomas), an explicit

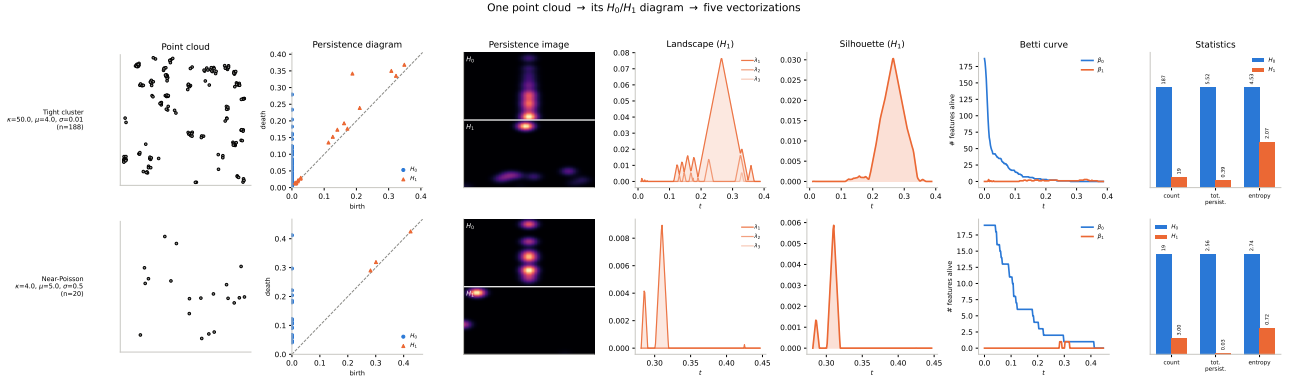


Figure 3: One point cloud $\rightarrow$ its H_0/H_1 persistence diagrams $\rightarrow$ the vectorizations compared in Table 1: persistence-image raster, landscape stack, silhouette curve, Betti curve, and persistence-statistics summary. Top row: the tight-cluster design ($\kappa = 50$, $\mu = 4$, $\sigma = 0.01$); bottom row: the near-Poisson design ($\kappa = 4$, $\mu = 5$, $\sigma = 0.5$). Statistics-panel bars are min-max normalized per statistic, with raw values printed above each bar, since count, total persistence, and entropy live on incomparable scales.

evaluation of the design choices at each stage, and a comparison of TDA-based vectorizations, on which there has been no formalized discussion in this setting.

Filtration. My starting point was the canonical Rips filtration, but it is built from inter-point distances alone: it is sensitive to the sparsest connection between two regions and does not register how densely those regions are occupied. Since the parameters of a point process act on local density, DTM_k seemed the richer choice, and the experiments confirmed this – Rips is the worst filtration arm on both processes, by a wide margin on nested Thomas (Table 1). Within DTM, error grows monotonically with k , consistent with a finer k -nearest neighbourhood retaining more of the local-density signal while a coarser one averages it away. I experimented with fusing several k scales to give the encoder fine and coarse summaries simultaneously; this helps on nested Thomas but not on Thomas, so the reported models use $k = 5$ for Thomas and fused $k \in \{5, 10, 15\}$ for nested Thomas. That this precisely is a per-process choice is a weakness I return to in Section 4.

Encoder, head, and ablations. For each choice of filtration I compute H_0 and H_1 persistence images. So on Thomas, which uses DTM_5 only, the model computes two diagrams per cloud, while Nested Thomas, which uses $\text{DTM}_{\{5,10,15\}}$ requires computing six diagrams (both homology dimensions for each of the three filtrations). All images are 64×64 . Each is passed through a small convolutional stack: three 3×3 blocks with channel widths (32, 64, 128), each followed by a ReLU and 2×2 max-pooling with spatial dropout ($p = 0.2$) between blocks. Each block is a CoordConv layer (Liu et al., 2018), concatenating two channels holding each pixel’s normalized coordinates before the convolution. I added this to the architecture because persistence image’s axes are not arbitrary – birth encodes local density and persistence encodes separation – so a pixel’s absolute location, not just its neighbourhood, is informative. The feature map is reduced by adaptive average pooling and projected to $E = 64$, $\log N(\mathbf{x})$ is appended, and the result is passed to an FCNN head with two hidden layers of width 64 and 32 (ReLU, dropout $p = 0.1$).

When three scales are used they share one encoder and their embeddings are concatenated. Every experiment uses AdamW (lr = 10^{-3} , weight decay 3×10^{-4}), batch size 32, up to 500 epochs with early-stopping patience 50.

Each of these choices was ablated one axis at a time against this configuration (Table 1). Notable, when multiple filtration sare used, shared encoder beats three independent ones, which is desirable due to the significant different in trainable parameters. H_0 alone is competitive while H_1 alone collapses to chance, so the merge structure carries most of the signal. Of the vectorizations, Figure 3 shows the four compared on a common diagram, and none of landscapes, silhouettes or Betti curves improved on persistence images on either process. Two further arms isolate where the signal lives: training on $\log N$ alone collapses to the chance level on both processes, while removing $\log N$ from the full model costs little on nested Thomas ($0.192 \rightarrow 0.203$) and is merely noisy on Thomas. The persistence images, not the point count, are doing the work.

Axis	Arm	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
Filtration	Rips (vec_multik, landscape)	0.178 ± 0.008	0.461 ± 0.016
	DTM ₅ / DTM ₁₅	0.125 ± 0.007 / 0.146 ± 0.006	0.205 ± 0.003 / 0.260 ± 0.017
	DTM _{5,10,15} (fused)	0.126 ± 0.008	0.192 ± 0.010^c
Homology	H_0 only / H_1 only	0.125 ± 0.006 / 0.900 ± 0.019	$0.280 \pm 0.250^\dagger$ / 0.932 ± 0.028
Encoder	shared / independent, concat	0.125 ± 0.007 / 0.134 ± 0.009	0.201 ± 0.010 / 0.222 ± 0.021
Vectorization	Persistence image	0.126 ± 0.008	0.192 ± 0.010^c
	Landscape	0.132 ± 0.008	0.250 ± 0.005
	Silhouette ($p = 1$)	0.165 ± 0.013	0.271 ± 0.012
	Betti curve	0.196 ± 0.014	0.290 ± 0.005
Features	log N removed	$0.217 \pm 0.278^\dagger$	0.203 ± 0.007
	Topology removed (log N only)	0.893 ± 0.015	0.948 ± 0.019

Table 1: Branch summary: every axis measured against the multi-scale, $H_0 + H_1$, CoordConv starting configuration, $n = 10$ seeds per arm unless noted. Vectorization rows are the native-encoder path. A dagger (†) marks an arm on which at least one seed failed to converge and collapsed to the chance level, so the mean is dominated by the failed seed(s). $^c_n = 3$. The DTM₅ / fused rows support the process-specific scale choice rather than ablating a single model.

3.3 Estimation performance and baseline comparison

I compare ParamNet against the classical and neural baselines of Section 1: minimum contrast fit to Ripley’s K and, separately, to the pair correlation function g , and `vihrs` – my re-implementation of the neural estimator of Vihrs (2022): a 1-D CNN over the edge-corrected, centred Ripley $L(r) - r$ curve plus $\log N(\mathbf{x})$, i.e. the pipeline of Section 3.2 with the persistence-image feature replaced by a single classical radial summary. All methods are run on the same clouds and splits, which are made by parameter vector so that no parameter value appears in both training and evaluation. Chance level is measured by retraining the default model on randomly permuted labels.

`vihrs` beats ParamNet on Thomas: its per-parameter losses ($\mathcal{L}^\kappa = 0.192$, $\mathcal{L}^\mu = 0.122$, $\mathcal{L}^\sigma = 0.029$) are each individually lower than ParamNet’s own (0.201, 0.134, 0.040; Table 3). So on this single-level process, the topological pipeline does not beat a hand-built radial summary. On nested Thomas, ParamNet gains an advantage on the density and count targets while `vihrs` maintains an edge on both scale parameters. $L(r) - r$ is built to resolve a characteristic radius directly, while a density-based topological feature is not, and it is only on the more complex two-level process that this trade-off favours the topological pipeline.

Both neural methods dominate minimum contrast on Thomas by roughly an order of magnitude, but for two different reasons. On K the estimator achieves losses of 0.365–0.399 for 6/10 seeds – close to `vihrs` – and 0.903–2.056 on the remaining 4/10, which is the near-Poisson flatness described in Section 1. On g every seed is worse than chance, indicating systematic bias rather than scattered non-convergence; the likely cause is my contrast exponent $c = 1$, set by analogy with Diggle (2003)’s K -based rule of thumb rather than tuned, where $c = 1/4$ is the value validated for aggregated patterns. I report the number but do not read it as a statement about contrast estimation on g in general.

Finally, parent intensity κ is the hardest target and the cluster scale σ the easiest, for every method on both processes and by a wide margin. This ordering holds regardless of estimator, so it looks like a property of the estimation problem itself rather than an artifact of any one feature set or architecture.

4 Discussion, limitations and future work

The results support three claims. Persistence images are informative about Neyman-Scott parameters: both processes sit roughly an order of magnitude below the label-shuffle ceiling, and the two “Features” arms of Table 1 place that signal in the images rather than in $\log N$. They are not better than a well-chosen radial summary on a single-level process: `vihrs` wins on all three Thomas parameters, and on both scale parameters of nested Thomas. The advantage is confined to the two-level process and to its density and count targets, which is what one would expect of a feature built from local density rather than from a characteristic radius. Two secondary observations: the ordering of parameter difficulty is stable across feature sets and architectures, so it reads as a property of the estimation problem; and the margin over minimum contrast is not a single effect, being seed-to-seed instability on K and a systematic bias traceable to an untuned contrast exponent on g . Neither number is a verdict on

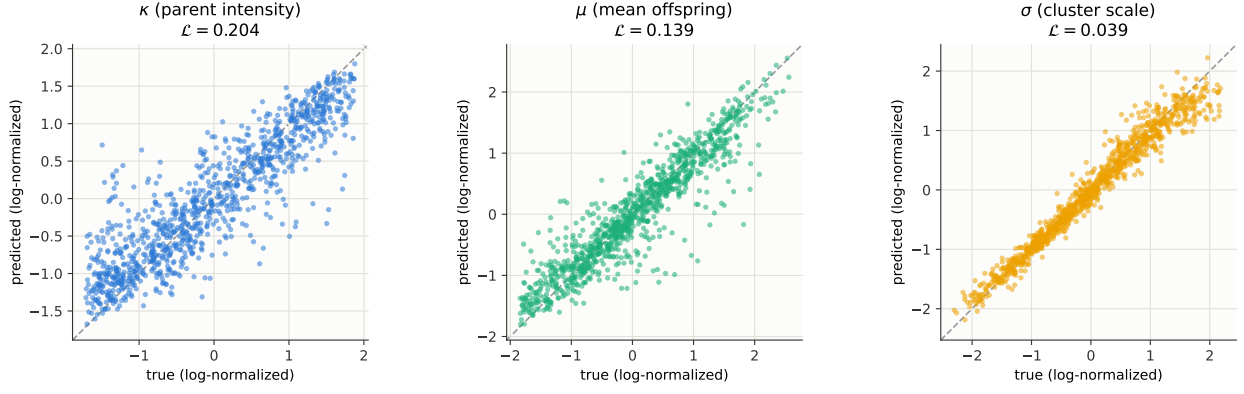


Figure 4: Predicted versus true parameter values, ParamNet (DTM₅, persistence image) on the shared Thomas test split, one seed, both axes in log-normalized units (log-transform then z-score against the train split, the space $\mathcal{L}$ is computed in); identity line dashed. The per-parameter difficulty ordering of Table 3 is visible directly: a tight diagonal band for σ , a wider cloud for κ , and in both cases the residual is spread around the identity line rather than a systematic offset. Intuition-building rather than publication-rigorous: one trained seed, no error bars or per-point uncertainty.

Method	Thomas $\mathcal{L}$	Nested-Thomas $\mathcal{L}$
ParamNet [‡]	0.125 ± 0.007	0.192 ± 0.010
vihrs ($L(r) - r$ CNN)	0.115 ± 0.007	0.217 ± 0.006
Minimum contrast, K	0.855 ± 0.682^a	— ^d
Minimum contrast, g	6.199 ± 0.339^b	— ^d
Chance-level reference (label shuffle)	1.003 ± 0.019	1.006 ± 0.020

Table 2: Estimator comparison on Thomas and Nested Thomas, $n = 10$ seeds, on the shared test split. [‡]ParamNet is process-specific: DTM₅ for Thomas, fused DTM_{5,10,15} for nested Thomas. ^a Minimum contrast on K is unstable across seeds rather than uniformly bad, and ^b minimum contrast on g is consistently worse than chance; both are discussed in the text. ^dNot run: no closed-form summary statistic for nested Thomas has yet been derived, so its absence is an outstanding experiment rather than an impossibility.

contrast estimation.

Limitations.

- **Multiple models.** The number of filtration scales is a per-process choice with no established rule beyond experimentation and comparison of validation losses, which undercuts the process-agnostic

(a) Thomas (κ parent intensity, μ offspring intensity, σ cluster scale)

Parameter	ParamNet	vihrs	Min. contrast, K	Min. contrast, g
$\mathcal{L}^\kappa$	0.201 ± 0.010	0.192 ± 0.013	1.373 ± 1.087	8.825 ± 0.485
$\mathcal{L}^\mu$	0.134 ± 0.007	0.122 ± 0.007	0.939 ± 0.758	6.334 ± 0.370
$\mathcal{L}^\sigma$	0.040 ± 0.004	0.029 ± 0.002	0.254 ± 0.203	3.437 ± 0.170

(b) Nested Thomas

Parameter	ParamNet ^c	vihrs
parent intensity	0.283 ± 0.023	0.320 ± 0.010
meta-offspring	0.402 ± 0.022	0.493 ± 0.022
meta-cluster scale	0.143 ± 0.006	0.132 ± 0.009
mean offspring	0.086 ± 0.004	0.110 ± 0.010
cluster scale	0.044 ± 0.000	0.028 ± 0.003

Table 3: Per-parameter loss $\mathcal{L}^j$ underlying the aggregate $\mathcal{L}$ of Table 2 (same methods, same seeds; ParamNet/nested is $n=3$). Minimum contrast has no nested-Thomas columns for the reason given there.

framing of Section 1. Always fusing would restore a single model at $3\times$ the persistence cost and 0.001 on Thomas, i.e. inside seed noise; attention over scales might be the fix.

- **Competitiveness.** Three alternative vectorizations were implemented and none improved on persistence images. This is evidence about these summaries under this architecture and this design distribution, not a general claim about vectorizations.
- **No advantage on Thomas.** `vihrs` wins on every parameter of the single-level process, and on both scale parameters of the nested one.
- **The design distribution is a prior.** The trained model approximates a posterior summary under it, so its ranges must cover the intended application domain.
- **Computational cost.** Persistence computation scales badly in n ; this is why the point budget of Section 3.1 is capped at 800. The inference cost per cloud is currently unmeasured.

Planned directions.

- The encoder architecture: aim to design a universal architecture that is process-agnostic.
- Inhomogeneous / non-stationary processes, where classical closed forms are least available and the topological approach has the most to gain. The comparator is the two-step procedure of Waagepetersen and Guan (2009), which needs the covariates observed and the intensity correctly specified; a simulation-trained estimator needs neither.
- Apply to real data. Galaxy catalogues if the running example of Section 1 is cosmological. The Barro Colorado Island tree data of Waagepetersen and Guan (2009) might be a second target: a small public data set already fitted with a Thomas process.

Part II

Hessian Spectra During Grokking

5 Grokking and motivation

Grokking is a training dynamic in which a model attains near-perfect training accuracy quickly, remains at chance-level test accuracy for an extended period, and then transitions sharply to high test accuracy long after the training loss has saturated. The delay is the object of interest: whatever separates the memorizing phase from the generalizing one is not visible in the training loss, which suggests the local geometry of the loss landscape rather than the loss value as the informative quantity. This project takes the Hessian spectrum as the description of that geometry, following Fort and Scherlis (2018), and estimates it by the Lanczos-based approach of Yao et al. (2020), which recovers spectral information without forming the matrix. I implemented that machinery directly rather than adopting the PyHessian library, so that the tracked observables and their numerical treatment remained under my control.

The current experiments use modular addition ($a + b \bmod p$) with a small decoder-only transformer trained with AdamW and weight decay large enough to penalize the pure-memorization solution. Training accuracy reaches $\sim 100\%$ within a few thousand optimization steps, while test accuracy remains near chance ($1/p$) for between hundreds and tens of thousands of further steps before rising above 90%. The length of that delay, and its dependence on weight decay and initialization scale, is the phenomenon this part of the review addresses.

TODO (figure): Train and test accuracy against optimization step on a log-step axis, for one or a few representative weight-decay values, with $\pm$ std bands over the ten seeds logged in `reports/wd_sweep.accuracy.curves.csv`. Takeaway: the shape of the phenomenon — immediate training saturation, a long flat test plateau at chance, then a sharp rise — established before any spectral quantity is introduced. Intuition-building rather than publication-rigorous: the seed bands are free since they are already logged, but one representative weight decay suffices and no significance testing is needed.

6 Spectral observables

For parameters $\theta \in \mathbb{R}^n$ and loss $\mathcal{L}(\theta)$, the Hessian is $\mathbf{H} = \nabla_{\theta}^2 \mathcal{L}(\theta) \in \mathbb{R}^{n \times n}$, symmetric with eigenvalues $\lambda_1 \leq \dots \leq \lambda_n$. This is the true Hessian rather than a Gauss–Newton or Fisher surrogate, so it may be indefinite: negative eigenvalues correspond to directions of negative curvature, which a positive semidefinite surrogate suppresses by construction. Rather than individual eigenvalues, eight scalar summaries of the full spectrum are logged per checkpoint: the top eigenvalue; a robust bulk/outlier threshold (`bulk_edge`) and the count of outliers above it; the trace; the spectral entropy and its exponential (`effective_rank`); the mass on meaningfully negative eigenvalues; and a conditioning proxy given by the ratio of top eigenvalue to bulk edge. Definitions and derivations are given in the companion note `reports/spectral_formalism.tex`.

7 Matrix-free spectral estimation

For a network with n in the 10^3 – 10^7 range, forming $\mathbf{H}$ explicitly is infeasible, so the computational primitive is the Hessian-vector product. Writing $g(\theta) = \nabla_{\theta} \mathcal{L}(\theta)$, Pearlmutter’s identity gives $\mathbf{H}v = \nabla_{\theta}(g(\theta)^{\top} v)$ from two backward passes. Supplying m such products to the Lanczos recurrence, initialized at a random unit vector, yields an m -dimensional tridiagonal reduction of $\mathbf{H}$ whose Ritz values and eigenvector weights form a moment-matching approximation to the spectral measure of $\mathbf{H}$ restricted to the corresponding Krylov subspace. Averaging over k independent random probe vectors — stochastic Lanczos quadrature — gives a pooled estimate of the spectral measure, and every observable of Section 6 is a functional of that pooled estimate. The Krylov depth m and probe count k are the two parameters trading compute against accuracy.

8 Estimator validation and (m, k) convergence

All eight observables were validated against two exact-reference fixtures with known ground-truth spectra: a small MLP whose Hessian is materialized in full, and an engineered quadratic-form fixture with a production-like ratio between the top eigenvalue and the bulk. The validation identified two estimator defects, distinct from mere under-convergence. First, the original `bulk_edge` and conditioning definitions used an unweighted median and MAD over the Ritz values, which was biased by 30–50% at every Krylov depth: Lanczos node placement is moment-matching rather than density-matching, so an unweighted median over-trusts the sparse nodes near the spectral edge. Second, the original `negative_mass` definition integrated mass immediately either side of the zero crossing and therefore measured Lanczos noise rather than curvature; it is now margin-thresholded. Both are corrected in the current code. Table 4 reports the resulting per-observable convergence requirements.

Observable	Governed by	Minimum m	Minimum k
trace	k (Monte Carlo, $\propto k^{-1/2}$)	irrelevant	≥ 100
top_eig	m	≥ 20	1
negative_mass (margin-thresholded)	m	≥ 30 –50	≥ 30
spectral_entropy / effective_rank	m	≥ 20 –30	≥ 30
bulk_edge / conditioning (post-fix)	m	≥ 75 –100	not yet swept

Table 4: Approximation accuracy against Lanczos depth m and probe count k , measured on the exact-reference fixtures (`reports/spectral_validation.md`). Production defaults are $m = 100$, $k = 100$, set by `bulk_edge/conditioning`, the deepest-converging pair.

This analysis is carried out entirely on synthetic fixtures of roughly 10^3 parameters. It has not yet been validated against the spectrum of a trained network on the modular-addition task; that check is listed among the next steps in Section 11.

9 Experiments

Three datasets appear across the project’s history. Two-spirals, a two-dimensional binary classification problem, was the setting in which the Hessian tooling was first built and tested, before a full architectural rewrite of the codebase. MNIST and FashionMNIST followed, with an MLP, once the

focus moved to grokking dynamics proper; an FGSM adversarial-robustness check was run alongside the curvature measurements then. Modular addition, with a small decoder-only transformer on $a + b \bmod p$, is where all current experiments run and is the source of every measured number reported below.

Two hyperparameters control the onset of grokking on this task, and both were swept with logged runs (AdamW, effectively full-batch, $p = 91, 150,000$ steps). Weight decay was swept from 0.45 to 1.0 in steps of 0.05, ten seeds per setting. Initialization scale was swept at $\alpha \in \{1, 8\}$, where every parameter is multiplied by the scalar α after standard initialization. The trends are monotone. At fixed α , larger weight decay produces an earlier generalization onset: the mean onset step falls from ~ 3150 at $wd = 0.45$ to ~ 1350 at $wd = 1.0$, under a first-crosses-twice-chance onset definition. At fixed $wd = 1.0$, $\alpha = 8$ roughly doubles the onset delay relative to $\alpha = 1$ (~ 2890 against ~ 1350 steps). This sweep did not log Hessian spectra during training: at this scale the cost of doing so alongside 240 training runs was prohibitive. That omission is the gap addressed in Section 11.

TODO (figure): Mean generalization onset step against weight decay, one line per initialization scale $\alpha \in \{1, 8\}$, log y -axis, seed spread shown as a band or as individual points. Takeaway: both control parameters in one picture — monotone decrease of onset with weight decay, and a roughly constant multiplicative offset between $\alpha = 8$ and $\alpha = 1$. Data source: the logged sweeps described above. Intuition-building rather than publication-rigorous: no fitted trend line or confidence interval, the ten seeds per setting shown only to indicate spread.

10 Current results and their status

The reproducible result currently supported by logged data is the sweep of Section 9: grokking onset on modular addition is systematically and monotonically controlled by weight decay and initialization scale. The remaining material in this section is either historical or not yet run on real data, and is separated accordingly.

Recalled spectral collapse (unevidenced). During the earlier MNIST-era phase of the project I observed what appeared to be a spectral collapse, with eigenvalues above roughly 1 disappearing around the grokking transition. This recollection is not currently supported by logged data, a saved figure, or a write-up anywhere in the current repository: it predates the architectural rewrite and was not retained. It is recorded here as an observation to be re-established, not as a demonstrated result.

Gauss–Newton decomposition (not retained). Before the same rewrite I worked on the decomposition $\mathbf{H} = \mathbf{G} + \mathbf{F}$, in which $\mathbf{G}$ depends only on the sensitivity of the network’s outputs to its parameters and is positive semidefinite, while $\mathbf{F}$ carries the signed prediction residuals; $\mathbf{F}$ is therefore the sole source of negative curvature, and $\mathbf{H} \rightarrow \mathbf{G}$ as the fit improves. On a small two-spirals MLP at initialization the decomposition held to machine precision ($\lambda_{\min}(\mathbf{G}) \approx -3 \times 10^{-17}$, decomposition residual $\approx 2 \times 10^{-17}$), and the effective rank of $\mathbf{G}$ was ≈ 3 against 337 total parameters, indicating persistent low-rank structure. This machinery was correct, but it predates the architectural rewrite and was not retained; the current codebase tracks the true Hessian only.

Weight-norm normalisation and the gate figure (implemented, not yet run on real data). Curvature scales as $\|\theta\|^{-2}$ under a uniform rescaling of the parameters, so raw trace and top-eigenvalue traces move whenever the weight norm does, independently of any change in the loss geometry. A normalisation by weight norm is implemented for this reason, and a corresponding two-panel diagnostic figure — test accuracy and weight norm above, weight-norm-corrected trace, top eigenvalue and effective rank below, with the onset step marked — is implemented end to end (`analysis/transitions.py`, `analysis/make_figures.py`) and verified against synthetic input. Neither has been run against a spectral parquet from an actual training run. This figure is the instrument that would test the recalled collapse above.

TODO (figure): The two-panel gate figure described above, once the first real spectral parquet exists. Upper panel: test accuracy and weight norm against optimization step, shared log-step axis, onset marked with a vertical rule. Lower panel: weight-norm-corrected trace, top eigenvalue and effective rank. Takeaway: whether any spectral observable moves at the onset once the trivial $\|\theta\|^{-2}$ scaling has been divided out. Data source: the checkpoints of Section 11. Intuition-building rather than publication-rigorous: one representative run and no error bars for a first look, though the claim it tests will eventually require several seeds.

11 Next steps

Returning to this project after a period away identified two gaps in the earlier work: no systematic accuracy analysis of the (m, k) approximation had been performed, and no observable had been normalized by parameter count or weight norm, despite the $\|\theta\|^{-2}$ scaling noted above. Both gaps are now closed at the level of tested code (Sections 8 and 10); neither has been applied to production data.

The immediate next step is mechanical. 9,465 checkpoints from the weight-decay sweep are already on disk; running `scripts/spectral_from_checkpoints.py` over a subset of them would produce the first spectral parquet the project has had from real training runs. At that point the gate figure becomes a result rather than validated-but-unrun code, and the recalled collapse of Section 10 can be tested with error bars at (m, k) settings whose accuracy is characterized.

Beyond that, Tuci et al. (2026) is the most promising direction to build on: it derives a generalization bound from the full Hessian spectrum and connects it explicitly to grokking, which is close to the question motivating this work. The stochastic Hessian-trace estimators of Liu et al. (2023) were read alongside the Gauss–Newton literature while the trace estimator of Section 7 was being built.

References

- H. Adams et al. Persistence images: a stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18(8):1–35, 2017.
- H. Anai, F. Chazal, M. Glisse, Y. Ike, H. Inakoshi, R. Tinarrage, and Y. Umeda. DTM-based filtrations. In *Topological Data Analysis*, Abel Symposia 15, pages 33–66. Springer, 2020.
- P. Bubenik. Statistical topological data analysis using persistence landscapes. *Journal of Machine Learning Research*, 16:77–102, 2015.
- F. Chazal, B. T. Fasy, F. Lecci, A. Rinaldo, and L. Wasserman. Stochastic convergence of persistence landscapes and silhouettes. In *Proc. 30th Annual Symposium on Computational Geometry*, pages 474–483, 2014.
- P. J. Diggle. *Statistical Analysis of Spatial Point Patterns*. Arnold, London, 2nd edition, 2003.
- S. Fort and A. Scherlis. The Goldilocks zone: towards better understanding of neural network loss landscapes. arXiv:1807.02581, 2018. Published at AAAI-19.
- R. Liu et al. An intriguing failing of convolutional neural networks and the CoordConv solution. In *Advances in Neural Information Processing Systems*, 2018.
- Y. Liu, S. Yu, and T. Lin. Regularizing deep neural networks with stochastic estimators of Hessian trace. arXiv:2208.05924, 2022. Published in *Neurocomputing*, 536:13–20, 2023.
- J. Møller and R. P. Waagepetersen. *Statistical Inference and Simulation for Spatial Point Processes*. Chapman & Hall/CRC, 2004.
- J. Reininghaus, S. Huber, U. Bauer, and R. Kwitt. A stable multi-scale kernel for topological machine learning. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 4741–4748, 2015.
- U. Tanaka, Y. Ogata, and D. Stoyan. Parameter estimation and model selection for Neyman-Scott point processes. *Biometrical Journal*, 50(1):43–57, 2008.
- M. Tuci, C. Korkmaz, U. Şimşekli, and T. Birdal. Generalization at the edge of stability. arXiv:2604.19740, 2026.
- Y. Umeda. Time series classification via topological data analysis. *Transactions of the Japanese Society for Artificial Intelligence*, 32(3):1–12, 2017.
- N. Vihrs. Using neural networks to estimate parameters in spatial point process models. arXiv preprint arXiv:2109.15056, 2022.
- R. Waagepetersen and Y. Guan. Two-step estimation for inhomogeneous spatial point processes. *Journal of the Royal Statistical Society: Series B*, 71(3):685–702, 2009.
- Z. Yao, A. Gholami, K. Keutzer, and M. Mahoney. PyHessian: neural networks through the lens of the Hessian. arXiv:1912.07145, 2020.
- J. H. T. Yip, A. Rouhiainen, and G. Shiu. Learning from topology: cosmological parameter estimation from the large-scale structure. *Machine Learning: Science and Technology*, 6:025063, 2025.